



API CLIENTS • BACKEND OPERATORS

PRO API MANUAL

Flask Pro API, auth, /run contract, preview, backtest

Open-source Pine Script evaluation framework



CONTENTS

01 Overview

02 App Lifecycle

03 Auth And Keys

04 Run

05 Runner

06 Optimize

07 Preview

08 Backtest

09 Chart Renderer

10 Runtime Bridge

11 Contract

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Pro API" description: "Flask Pro API — /run with mode=auto, /optimize strategy search, plot_meta/fill, alerts + L2 webhooks, preview, backtest, and shared evaluate contract."

PRO API

The PYNE **Pro API** is a Flask HTTP service that runs Pine scripts over caller-supplied OHLCV, returns plots / series / `plot_meta` (incl. `fill` bands) / strategy events / drawings / **alerts**, optional **L2 alert webhooks**, and gates thumbnail + backtest routes behind API keys. Default evaluate path is `mode=auto` (compile when safe, interpret on fallback). It is intentionally **not** the language server: different auth, scaling, and failure domains.

Abstract

Where the LSP answers *static* editor questions, the Pro API answers *dynamic* evaluate questions:

```
POST JSON { script, data: OHLCV[], mode?, libraries?, webhook_url?, ... }
→ pynescrypt.runtime.Runtime.run (Pro API re-export; mode default: auto)
→ { plots, series, plot_meta, events, drawings, alerts,
    mode, auto_backend?, compile_fallback_reason?,
    script_id, run_id, alert_forward?, ... }
```

Free-tier evaluate (`POST /run` , `POST /run/batch`) validates bodies with hand-rolled schemas, enforces a 5 MB body cap, and restricts CORS. Optional **bar/script/rate/concurrency** guards (`backend/middleware/free_limits.py`) are **off unless** `FREE_TIER_LIMITS` is `1 / true / yes / on` . `GET /` and `GET /health` are the same unauthenticated readiness payload (`features.free_tier_limits` reports the switch). Optional **L2 webhooks** `POST` last-bar `alert()` / `alertcondition()` firings to `webhook_url` or env `ALERT_WEBHOOK_URL` (see [Alerts](#)). Pro routes under `/preview/*` and `/backtest/*` use `track_usage` (Bearer / ApiKey / query key). Chart PNGs are matplotlib Agg renders encoded as base64.

The same **evaluate contract** is mirrored by **pyne-worker** so AXIS and HOOX can swap hosts without redesigning payloads — see [contract](#).

Conceptual model

```
flowchart TB
    CLIENT[AXIS / CLI / curl] --> FLASK[backend.app Flask]
    FLASK --> AUTH[middleware auth + schemas + free_limits]
    FLASK --> RUN["/run · /run/batch · /ws/run"]
    FLASK --> OPT["/optimize"]
    FLASK --> PREV["/preview/*"]
    FLASK --> BT["/backtest/*"]
    FLASK --> GIT["/api/git/oauth/* optional"]
    RUN --> RT[pynescrypt.runtime.Runtime]
    RUN --> WH[alert_forwarder L2]
    PREV --> CHART[chart_renderer]
    BT --> BACK[services.backtest]
    RT --> EVAL[CustomEvaluator + PineSeries]
    RT --> CMP[compiler.engine when auto/compile]
    RT --> AST[parse ASDL]
```



Interface surface

Method	Path	Auth	Role
GET	/ · /health	none	Health + endpoint map + compile cache section
POST	/run	none (free)	Single-script evaluate (mode default auto ; libraries[] ; alerts + optional webhooks)
POST	/run/batch	none (free)	≤8 scripts, shared OHLCV
POST	/scripts · /cron/*	optional runner	Hosted scripts + bar-close — off unless PYNE_RUNNER=1 (runner)
POST	/optimize	none (free)	Strategy input.* search — N interpret Runtime runs (docs)
WS	/ws/run	none (free)	Same evaluate contract over JSON frames (flask-sock optional)
POST	/preview/chart	API key + usage	Line / OHLCV PNG
POST	/preview/indicator	API key + usage	Expression series PNG
POST	/backtest/quick	API key + usage	Quick strategy metrics + equity PNG
POST	/auth/create_key	admin decorator	Mint pyn_... key (fail-closed ADMIN_TOKEN)
GET	/auth/usage	API key	Usage counters
POST	/auth/validate	none (body key)	Validate without consuming quota
POST	/compile/prewarm	none (free, rate-gated)	Product warm-compile — skip cold Numba JIT
POST	/lsp/*	none (free)	AXIS editor completion / hover / diagnostics
POST	/api/git/oauth/device/*	none (device flow)	Optional GitHub/GitLab device OAuth for SPA git (blueprint)

/run response highlights

Field	Role
series / plots / plot_meta	Bar values + display meta (color, linewidth, kind , fill band refs)
events	Strategy broker events
drawings	Drawing registry / compile __drawings
alerts	Structured alert() / alertcondition() firings
mode / auto_backend	Effective path (interpret compile)
compile_fallback_reason	Why auto chose interpret (eligibility or compile error)
alert_forward	Present when a webhook URL was configured
error_kind	On failure: parse compile runtime data order mode

Local runner: make run → python -m backend.app (default 127.0.0.1:5002).



Tracks in this tab

1. [App lifecycle](#) — Flask app, CORS, limits, blueprints
2. [Auth and keys](#) — tiers, stores, decorators
3. [Run](#) · [Optimize](#) · [Preview](#) · [Backtest](#)
4. [Chart renderer](#)
5. [Runtime bridge](#) — `Runtime` / `series` / `evaluator` glue
6. [Contract](#) — shared evaluate schema across Flask and workers

Internals (map)

Path	Role
<code>backend/app.py</code>	App object, <code>/run*</code> , <code>/health</code> , <code>/compile/prewarm</code> , <code>/auth*</code> , CORS
<code>backend/api/preview.py</code>	Preview + backtest blueprints
<code>backend/api/git_oauth.py</code>	Optional device OAuth for SPA git
<code>backend/api/lsp_http.py</code>	Free AXIS editor LSP-HTTP
<code>backend/middleware/*</code>	Auth, schemas, <code>free_limits</code> , optional Redis/SQLite stores
<code>src/pynescrypt/runtime/host.py</code>	Package SoT bar-loop (<code>mode=auto</code> , alerts pack)
<code>backend/runtime.py</code>	Compat re-export of the package host
<code>backend/alert_forwarder.py</code>	L2 HTTP webhook delivery
<code>backend/evaluator.py</code> / <code>backend/series.py</code>	Compat shims
<code>backend/services/*</code>	Charts + backtest simulation

See also

- [Evaluate scripts \(end user\)](#)
- [Pro API usage guide](#)
- [Alerts](#)
- [Runtime hub](#)
- [LSP hub](#) — editor path, not HTTP

APP LIFECYCLE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "App Lifecycle" description: "Flask application setup: MAX_CONTENT_LENGTH, CORS policy, blueprints, error handlers, and process entry."

APP LIFECYCLE

Abstract

`backend/app.py` constructs a single global `Flask` app, applies security-minded defaults (body size, CORS allowlist), registers free and Pro routes, and exposes a module `__main__` for local development. There is no multi-app factory pattern today — import `app` for WSGI (gunicorn) or run the module for the built-in server.



Conceptual model

```
flowchart TB
    BOOT[import backend.app] --> CFG[MAX_CONTENT_LENGTH + CORS]
    CFG --> ROUTES[register / · /health · /run · /auth · /compile]
    ROUTES --> BP[preview + backtest + lsp + git_oauth]
    BP --> ERR[404 / 500 JSON handlers]
    ERR --> SERVE{entry}
    SERVE -->|gunicorn| WSGI[app:app]
    SERVE -->|python -m backend.app| DEV[HOST/PORT env]
```

Interface surface

Process entry

```
make run
# equivalent
python -m backend.app
```

Environment:

Variable	Default	Role
HOST	127.0.0.1	Bind address (localhost-first for dev)
PORT	5002	Listen port
ALLOWED_ORIGINS	https://pynescrypt.ai , https://app.pynescrypt.ai , localhost regex	CORS origins (comma-separated; regex allowed). * / any opens all. Code always appends localhost, private-LAN, and product (hoox.sh / pynescrypt.ai / pynescrypt- axis.pages.dev) regexes unless the value is *
API_KEY_STORE	/data/api_keys.json	JSON key-store path (auth.APIKeyStore default)
ADMIN_TOKEN	unset	Fail-closed admin minting (see auth). When set, hosted-runner writes also require it
PYNE_RUNNER	off	Optional hosted script registry + /cron/run (runner)
PYNE_RUNNER_SCHEDULER	off	In-process bar-close poll (requires PYNE_RUNNER)
PYNE_RUNNER_POLL_SECONDS	60	Scheduler interval
PYNE_RUNNER_DB	/data/runner.db or temp	SQLite registry path

debug=False always on the dev runner.

Health

GET / and GET /health are the same handler. AXIS Settings probes /health . Payload includes compile-cache diagnostics:



```
{
  "status": "healthy",
  "service": "pynescrypt-pro-api",
  "version": "1.0.0",
  "timestamp": 0,
  "websocket": true,
  "features": {
    "alerts": true,
    "alert_webhooks": true,
    "warm_compile": true,
    "default_run_mode": "auto",
    "script_runner": false,
    "script_runner_scheduler": false
  },
  "compile": {
    "has_numba": true,
    "disk_cache_enabled": true,
    "prewarm_enabled": true,
    "default_mode": "auto"
  },
  "endpoints": { "...": "..." }
}
```

`version` here is the **API service** label (`1.0.0`), not the `hoox-pyne` package version.

Hard limits

- `MAX_CONTENT_LENGTH = 5 * 1024 * 1024` — reject oversized bodies before JSON parse fills memory (audit 2026-07-05 / S1).
- Free compute is optionally gated by `backend/middleware/free_limits.py` when `FREE_TIER_LIMITS` is truthy (default **off**): max bars (`FREE_MAX_BARS` , 5000), script chars (`FREE_MAX_SCRIPT_CHARS` , 256 KiB), per-IP rate (`FREE_RATE_LIMIT` / `FREE_RATE_WINDOW_SEC` , 60 / 60s), concurrency (`FREE_MAX_CONCURRENT` , 4). Chart/ `mock` data sources only. Production compose sets `FREE_TIER_LIMITS=1` .
- CORS methods: `GET` , `POST` , `OPTIONS` , `HEAD` .
- Allow headers: `Content-Type` , `Authorization` , `X-Admin-Token` , `Accept` .
- `supports_credentials=False` .

Localhost regex used in the default origin list:

```
^https?:/(?:localhost|127\.0\.0\.1)(?:\:d+)?$
```

Free CORS prefixes (`/` , `/health` , `/run` , `/compile` , `/lsp/` , `/ws/`) reflect the request `Origin` even when it is not in `ALLOWED_ORIGINS` so AXIS VPS UI → local pyne preflight succeeds.

Same-origin / no `Origin` header (curl, server-to-server) remains usable under flask-cors behavior.

Blueprints

```
app.register_blueprint(preview_bp)    # url_prefix=/preview
app.register_blueprint(backtest_bp)   # url_prefix=/backtest
app.register_blueprint(lsp_bp)        # url_prefix=/lsp
app.register_blueprint(git_oauth_bp)  # /api/git/oauth/*
```



Preview + backtest live in `backend/api/preview.py` . Optional `flask-sock` registers `WS /ws/run` when installed.

Error handlers

Code	Body code	Message
404	<code>NOT_FOUND</code>	Endpoint {path} not found.
500	<code>INTERNAL_ERROR</code>	Internal server error.

Both return JSON with `status: "error"` .

Internals

Path	Role
<code>backend/app.py</code>	App object, free routes, auth routes, CORS, health
<code>backend/api/preview.py</code>	Pro preview + backtest blueprints
<code>backend/api/lsp_http.py</code>	Free AXIS LSP-HTTP
<code>backend/middleware/schemas.py</code>	Request validation for <code>/run*</code> and auth
<code>backend/middleware/free_limits.py</code>	Optional bar / script / rate / concurrency caps (<code>FREE_TIER_LIMITS</code>)
<code>backend/requirements.txt</code>	Flask, flask-cors, numpy, matplotlib, ...

Recommended production: gunicorn/uvicorn-class WSGI with multiple workers; for multi-worker key consistency prefer SQLite/Redis stores over the default in-memory/file singleton assumptions — see [auth](#).

Invariants and edge cases

1. **Import side effects:** creating `app` configures CORS immediately from env.
2. **Body too large** → Flask 413 before route logic (not custom JSON).
3. **Unknown fields** on schema-validated routes → `UNKNOWN_FIELDS 400` (`/run` , auth); preview routes currently use looser `get_json()` (schemas exist but are not always applied in handlers).
4. **Dev bind default is loopback** — set `HOST=0.0.0.0` only when intentional.

Worked example — smoke test

```
curl -s http://127.0.0.1:5002/health | jq '.status, .features.default_run_mode'
# "healthy"
# "auto"
```

Failure modes

Symptom	Cause
Browser CORS errors	Origin not in <code>ALLOWED_ORIGINS</code>
413 on large OHLCV	Exceeded 5 MB — downsample or paginate
Import errors for matplotlib	Missing backend deps — <code>pip install -r backend/requirements.txt</code>



See also

- [Auth and keys](#)
- [Run endpoint](#)
- [Docker / CI](#)

AUTH AND KEYS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Auth and Keys" description: "API key model, tiers, require_api_key / track_usage decorators, JSON/SQLite/Redis stores, and admin create_key."

AUTH AND KEYS

Abstract

Pro routes authenticate with a raw **API key** presented as `Authorization: Bearer ...`, `Authorization: ApiKey ...`, or `?api_key=`. Keys are minted as `pyn_` + url-safe secret, stored with a short `key_id` and SHA-256 hash metadata, and tracked against **monthly-ish call limits** by tier. Free `/run` does not require a key.



Conceptual model

```
flowchart LR
    REQ[HTTP request] --> HDR{Authorization / query}
    HDR --> VAL[store.validate_key]
    VAL -->|miss| U401[401 UNAUTHORIZED]
    VAL -->|hit + over limit| U429[429 RATE_LIMITED]
    VAL -->|ok| G[g.api_key]
    G --> H[handler]
    H --> TRACK[increment_calls on success]
```

Interface surface

Tiers (`_TIER_LIMITS`)

Tier	calls_limit
free	0 (treated as unlimited in <code>is_rate_limited</code> — limit 0 means “no cap”)
hobby	5000
pro	50000
team	200000
enterprise	inf

`calls_remaining()` returns `inf` when limit is 0.

Endpoints

POST `/auth/create_key`

Body schema `CREATE_KEY_SCHEMA` : optional `tier` (default `"hobby"`).

Response:

```
{
  "status": "success",
  "api_key": "pyn_...",
  "key_id": "12hexchars",
  "tier": "hobby",
  "message": "Store this API key securely. It will not be shown again."
}
```

Decorated with `@require_admin_token` :

- Requires non-empty `ADMIN_TOKEN` env (unset → **403**, minting disabled)
- Client must send matching `X-Admin-Token` (or `Authorization: Bearer ... / AdminToken ...`)
- Comparison uses `hmac.compare_digest`

GET `/auth/usage`

`@require_api_key` — returns `key_id`, `tier`, and usage block (`calls_used`, `calls_limit`, `calls_remaining`, timestamps).



POST /auth/validate

Body: `{ "api_key": "..." }` — validates without incrementing usage. Returns tier info, `active`, `rate_limited`.

Decorators

Decorator	Behavior
<code>require_api_key</code>	Resolve key → 401 / 429 or set <code>g.api_key</code>
<code>track_usage</code>	Wraps <code>require_api_key</code> ; increments calls when response status < 400 (tuple responses) or always for bare Response
<code>require_admin_token</code>	Fail-closed admin gate (<code>ADMIN_TOKEN</code> + <code>X-Admin-Token</code>)

Client presentation

```
curl -H "Authorization: Bearer $API_KEY" \  
  -H "Content-Type: application/json" \  
  -d @body.json \  
  http://127.0.0.1:5002/preview/chart
```

Internals

APIKey dataclass

Fields: `key_id`, `key_hash`, `tier`, `calls_used`, `calls_limit`, `created_at`, `last_used`.

Helpers: `is_active()` (always True today), `is_rate_limited()`, `get_tier_info()`, `increment_calls()`.

Stores

Module	Use
<code>middleware/auth.py</code> → <code>APIKeyStore</code> + <code>get_key_store()</code>	Facade; selects backend via <code>STORE_BACKEND</code>
<code>middleware/key_store_sqlite.py</code>	Hash-only rows, WAL, multi-worker friendly
<code>middleware/key_store_redis.py</code>	Hash-only Redis; multi-replica

`get_key_store()` backends (`STORE_BACKEND`):

Value	Persistence
<code>json</code> (default)	File at <code>API_KEY_STORE</code> (default <code>/data/api_keys.json</code>); hash-only object keys. Legacy files keyed by the raw <code>pyn_...</code> secret are migrated on load and the raw secret is dropped
<code>sqlite</code>	<code>API_KEY_STORE_SQLITE</code> (or <code>.db</code> beside JSON path); hash-only
<code>redis</code>	<code>REDIS_URL</code> required; hash-only

Prod compose defaults to `sqlite` on `/data/api_keys.db` so gunicorn workers share state.

Key minting:



```
raw_key = "pyn_" + secrets.token_urlsafe(32)
key_id = sha256(raw_key).hexdigest()[:12]
key_hash = sha256(raw_key).hexdigest()
```

Schemas

`CREATE_KEY_SCHEMA`, `VALIDATE_KEY_SCHEMA` in `middleware/schemas.py` — strict types, reject unknown fields.

Invariants and edge cases

1. `/run` is **unauthenticated** — rate-limit at reverse proxy if exposed publicly.
2. **Limit 0 means unlimited**, not “zero calls” — free tier limit value is 0 with that semantics.
3. **Usage increments only after handler returns** — failed auth never bills; handler 4xx under `track_usage` skips increment when returned as `(response, status)`.
4. **Revocation** exists on the store API (`revoke_key`); no public HTTP revoke route in `app.py`.
5. **Admin minting is fail-closed** — unset `ADMIN_TOKEN` or wrong `X-Admin-Token` → 403.

Worked example

```
# create (requires ADMIN_TOKEN env + matching header)

curl -s -X POST http://127.0.0.1:5002/auth/create_key \
  -H "X-Admin-Token: $ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"tier": "hobby"}'

# validate
curl -s -X POST http://127.0.0.1:5002/auth/validate \
  -H 'Content-Type: application/json' \
  -d '{"api_key": "pyn_..."}'
```

Failure modes

Code	Meaning
<code>UNAUTHORIZED</code>	Missing/invalid key
<code>RATE_LIMITED</code>	<code>calls_used >= calls_limit</code> (finite limits)
<code>FORBIDDEN</code>	<code>ADMIN_TOKEN</code> unset or <code>X-Admin-Token</code> mismatch (fail-closed)
<code>INVALID_TIER</code>	<code>create_key</code> with unknown tier string
<code>INVALID_KEY</code>	<code>validate</code> endpoint miss

See also

- [App lifecycle](#)
- [Preview](#)
- [Security \(DevOps\)](#)

RUN

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "POST /run" description: "Free evaluate endpoints: single-script /run and multi-script /run/batch over shared OHLCV."

POST `/run` **AND** `/run/batch`

Abstract

`/run` is the primary free evaluate entry: accept Pine source + bar list, optionally wire `request.*` data sources, execute via package `pynescrypt.runtime.Runtime` (Pro API uses `backend.runtime` re-export), and return plots/series/events/drawings/alerts. `/run/batch` runs up to eight scripts on the **same** OHLCV (AXIS multi-indicator), isolating per-script errors so one failure does not discard siblings. Optional **free-tier guards**



(`FREE_TIER_LIMITS` , default off) enforce bar/script caps, rate limit, concurrency, and chart/mock-only data (`backend/middleware/free_limits.py`). Optional **L2 webhooks** POST last-bar (default) `alert()` / `alertcondition()` firings to `webhook_url` or env `ALERT_WEBHOOK_URL` (SSRF-safe).

Conceptual model

```
sequenceDiagram
    participant C as Client
    participant A as Flask /run
    participant V as validate RUN_SCHEMA
    participant D as resolve_request_sources
    participant R as Runtime.run

    C->>A: POST JSON
    A->>V: schema + required fields
    V->>D: optional data_source
    D->>R: script, ohlcv, mode
    R-->>C: success payload or EXECUTION_ERROR
```

Interface surface

POST `/run`

Request (`RUN_SCHEMA`)

Field	Type	Required	Default	Notes
<code>script</code>	string	yes		Pine source
<code>data</code>	list	yes		OHLCV bar objects
<code>symbol</code>	string	no	<code>"CHART"</code>	Syminfo / request wiring
<code>data_source</code>	string	no	<code>" "</code>	<code>mock</code> / <code>ccxt</code> / <code>yahoo</code> / ...
<code>data_options</code>	object	no	<code>{}</code>	Provider options
<code>mode</code>	string	no	<code>"auto"</code>	<code>"interpret"</code> <code>"compile"</code> <code>"auto"</code>
<code>inputs</code>	object	no	<code>{}</code>	Pine <code>input.*</code> overrides by title
<code>profiler</code>	bool	no	<code>false</code>	Per-line timing on interpret
<code>webhook_url</code>	string	no	<code>" "</code>	L2 alert webhook (overrides <code>ALERT_WEBHOOK_URL</code>)
<code>forward_alerts</code>	bool	no	<code>true</code>	When false, skip outbound webhook
<code>alert_last_bar</code>	bool	no	<code>true</code>	Webhook only last OHLCV bar firings
<code>alert_batch</code>	bool	no	<code>true</code>	One batch POST vs per-alert
<code>libraries</code>	list	no	<code>[]</code>	<code>[{namespace, name, version, source}]</code> — AXIS git-publish emulator (max 32)
<code>timeout_seconds</code>	number	no	omit / <code>null</code>	Optional interpret wall-clock budget. Passed to <code>Runtime.run</code> only when set and <code>> 0</code> . Omit = no timeout.

Query `?mode=` is accepted when the body omits `mode` .



Unknown fields → UNKNOWN_FIELDS 400. Empty script/data still checked after schema with NO_SCRIPT / NO_DATA .
import and any request. token are compile-ineligible; mode=auto falls back to interpret and keeps libraries
on that path.

Bar shape (runtime expectation):

```
{ "time": 0, "open": 1, "high": 1, "low": 1, "close": 1, "volume": 1 }
```

Optional bid / ask per bar update runtime bid/ask state.

Success response

```
{
  "status": "success",
  "plots": [],
  "series": {},
  "plot_meta": {},
  "events": [],
  "drawings": [],
  "alerts": [],
  "script_id": "16hex",
  "run_id": "16hex",
  "count": 0,
  "mode": "compile",
  "auto_backend": "compile",
  "data_source": "chart",
  "alert_forward": {
    "forwarded": 0,
    "failed": 0,
    "filter": "last_bar",
    "url": "https://hooks.example.com/pine",
    "batch": true
  }
}
```



Field	Meaning
plots	Primary series (first plot) — backward compatible list
series	Named multi-plot map title → values per bar
plot_meta	Per-title color, linewidth, index
events	Strategy events with <code>script_id</code> / <code>run_id</code> stamps
drawings	Exported line/label/box registry for AXIS
alerts	<code>alert()</code> / true <code>alertcondition()</code> firings (interpret; empty on compile)
alert_conditions	Optional full condition evaluations (when present)
alert_forward	Present when a webhook URL was configured; delivery summary
script_id	<code>sha256(source)[:16]</code>
run_id	Per- Runtime instance id
auto_backend	<code>compile</code> <code>interpret</code> when requested <code>mode</code> was <code>auto</code>
compile_fallback_reason	Why <code>auto</code> used interpret (eligibility or compile error)
object_mode / compile_cached / compile_ms	Compile diagnostics when that path ran
nopython_fallback_reason	Numeric JIT failed; engine re-emitted object mode (still compile)

Errors

HTTP	code	When
400	<code>MISSING_FIELD</code> / <code>INVALID_FIELD</code> / <code>UNKNOWN_FIELDS</code> / <code>INVALID_BODY</code>	Schema
400	<code>NO_SCRIPT</code> / <code>NO_DATA</code>	Empty after defaults
400	<code>DATA_SOURCE_ERROR</code>	<code>resolve_request_sources</code> failed
400	<code>WEBHOOK_URL_BLOCKED</code>	<code>webhook_url</code> is private/loopback/metadata (SSRF denylist)
403	<code>DATA_SOURCE_FORBIDDEN</code>	Free path rejected live provider (<code>ccxt</code> / <code>yahoo</code> / ...) — only when <code>FREE_TIER_LIMITS</code> is on
413	<code>TOO_MANY_BARS</code> / <code>SCRIPT_TOO_LARGE</code>	Exceeded <code>FREE_MAX_BARS</code> (5000) or <code>FREE_MAX_SCRIPT_CHARS</code> (256 KiB) — only when <code>FREE_TIER_LIMITS</code> is on
429	<code>RATE_LIMITED</code> / <code>TOO_MANY_REQUESTS</code>	Free IP rate (60 / 60s) or concurrency (4) — only when <code>FREE_TIER_LIMITS</code> is on
500	<code>EXECUTION_ERROR</code>	Runtime returned <code>error</code> key; body may include <code>error_kind</code> / <code>error_type</code> / <code>error_bar</code>

POST /run/batch

Request (`RUN_BATCH_SCHEMA`)



Field	Type	Required	Notes
<code>scripts</code>	list	yes	strings or <code>{id, script}</code> objects
<code>data</code>	list	yes	Shared OHLCV
<code>symbol</code> / <code>data_source</code> / <code>data_options</code> / <code>mode</code> / <code>profiler</code>	optional	same as <code>/run</code> (no <code>inputs</code>)	
<code>libraries</code>	list	no	same AXIS git-publish list as <code>/run</code> (max 32; applied to every script)
<code>timeout_seconds</code>	number	no	same optional interpret budget as <code>/run</code> (applied to every script)
<code>webhook_url</code> / <code>forward_alerts</code> / <code>alert_last_bar</code> / <code>alert_batch</code>	optional	same L2 webhook fields as <code>/run</code> (applied per script result)	

Hard cap: `RUN_BATCH_MAX_SCRIPTS = 8` → `TOO_MANY_SCRIPTS`.

Response

```
{
  "status": "success" | "partial",
  "results": [ { "id": "...", "status": "success|error", ... } ],
  "count": 2,
  "ok": 1,
  "data_source": "chart"
}
```

HTTP **200** for partial success (per-script errors inside `results`). Envelope validation failures still 400.

Internals

```
runtime = Runtime(symbol=str(symbol))
result = runtime.run(script, ohlcv, data_feed=..., data_provider=..., mode=...)
```

See [runtime bridge](#) for bar-loop details and compile mode.

Paths: `backend/app.py` (`run_pine_script`, `run_pine_script_batch`, `compile_prewarm`, `WS/ws/run`), `backend/middleware/schemas.py`, `backend/middleware/free_limits.py`, `backend/alert_forwarder.py`.

Invariants and edge cases

1. **No API key** on these routes — free-tier guards apply only when `FREE_TIER_LIMITS` is on (see [Pro API usage](#)); protect further at the edge if needed.
2. **Batch isolates exceptions** per script (`try/except` around `runtime.run`).
3. **Whitespace-only `data_source`** coerced to `None` → chart default; free paths reject live providers (`ccxt/yahoo/...`) when `FREE_TIER_LIMITS` is on.
4. **Compile mode** may omit alert side effects (`alerts: []`); use `interpret/ auto` for alerts.
5. Schema rejects extras — clients must not send AXIS-only fields without updating schema.



6. **Webhook delivery is best-effort** — evaluate still returns `status: success` if the hook fails; check `alert_forward`.
7. Structured errors may include `error_kind` (`parse|compile|runtime|data|order|mode`), `error_type`, `error_bar`. `timed_out` is set on the HTTP body when Runtime exceeds `timeout_seconds`.

Worked example

```
curl -s http://127.0.0.1:5002/run \
-H 'Content-Type: application/json' \
-d '{
  "script": "@version=6\nindicator(\"t\")\nplot(close)",
  "data": [
    {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
    {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
  ],
  "symbol": "TEST:DEMO"
}'
```

Failure modes

Symptom	Cause
UNKNOWN_FIELDS	Typo'd property (e.g. <code>ohlcv</code> instead of <code>data</code>)
Parse errors in message	Invalid Pine — same engine as CLI
Batch <code>status: partial</code>	At least one script failed; inspect <code>results[i].message</code>

POST /compile/prewarm

Free readiness hook (same rate/concurrency gates). Body optional:

```
{ "scripts": ["@version=6\nindicator(\"x\")\nplot(close)"], "force": false }
```

Caps at 16 sources. Returns `has_numba`, `builtins_warmed`, `scripts_ok` / `scripts_failed`, `prewarm_ms`. Does **not** execute on OHLCV.

WS /ws/run

When `flask-sock` is installed: JSON text frames `{type: "run", id, script, data, mode?}`. Reply `{type: "result", ...}` or `{type: "error"}`. Ping/pong supported. Same `execute_run_payload` as HTTP `/run`.



Alert webhooks (L2)

```
# Server default (optional)

curl -s http://127.0.0.1:5002/run \
-H 'Content-Type: application/json' \
-d '{
  "script": "//@version=6\nindicator(\"a\")\nif bar_index == last_bar_index\n  alert(\"fire\")\n"mode": "interpret",
"webhook_url": "https://hooks.example.com/pine",
"data": [
  {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
  {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
]
}'
```

Full engine rules and batch payload shape: [Alerts](#).

See also

- [Alerts](#)
- [Contract](#)
- [Runtime bridge](#)
- [App lifecycle](#)

RUNNER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Hosted runner" description: "Optional Flask/VPS/container script registry + bar-close scheduler (PYNE_RUNNER). Off by default."

HOSTED RUNNER

Abstract

The Pro API is request/response: AXIS (or curl) POSTs bars, gets `alerts[]` in that reply, and nothing runs after the client disconnects. `PYNE_RUNNER` is an optional Flask-side loop that matches pyne-worker's "upload then keep going" model on **local, Docker, VPS, and Cloudflare Containers**.

Off unless `PYNE_RUNNER=1` (or `true` / `yes` / `on`). Routes then 404 with `RUNNER_DISABLED`. Health reports `features.script_runner`.



This is **not** HOOX order routing. Alerts leave via the same L2 webhooks as `POST /run`. Strategy events stay on the tick JSON unless you forward them yourself.

Conceptual model

```
flowchart TB
    C["C[curl / AXIS / cron] -->|POST /scripts| REG[SQLite registry]"]
    SCH["SCH[scheduler or POST /cron/run] --> REG"]
    P["P --> P[get_provider mock/yahoo/ccxt]"]
    RT["RT --> RT[Runtime.run]"]
    WH["WH[alert_forwarder]"]
    C --> SCH
    SCH --> P
    P --> RT
    RT --> WH
```

Each tick fetches OHLCV for the script's `symbol` / `timeframe` / `data_source`. If last bar time did not advance, the job is skipped. Otherwise a full `Runtime.run` (same evaluate contract as `/run`) and last-bar alert webhooks.

Interface surface

Environment

Variable	Default	Role
<code>PYNE_RUNNER</code>	off	Master switch — HTTP registry + <code>/cron/run</code>
<code>PYNE_RUNNER_SCHEDULER</code>	off	In-process poll loop (needs <code>PYNE_RUNNER</code>)
<code>PYNE_RUNNER_POLL_SECONDS</code>	60	Poll interval (floor 5s)
<code>PYNE_RUNNER_DB</code>	<code>/data/runner.db</code> if <code>/data</code> exists	SQLite file
<code>ADMIN_TOKEN</code>	unset	When set, writes require <code>X-Admin-Token</code>

Gunicorn multi-worker: every worker may start a scheduler thread; ticks take a flock on `PYNE_RUNNER_DB.lock`. Prefer `GUNICORN_WORKERS=1` when the scheduler is on.

Routes (`PYNE_RUNNER=1`)

Method	Path	Auth	Role
POST	<code>/scripts</code>	admin if <code>ADMIN_TOKEN</code>	Deploy / replace
GET	<code>/scripts</code>	none	List (no source)
GET	<code>/scripts/:id</code>	none	Source + cron state
DELETE	<code>/scripts/:id</code>	admin if <code>ADMIN_TOKEN</code>	Remove
GET	<code>/cron/jobs</code>	none	Scripts + last tick
PUT	<code>/cron/jobs</code>	admin if <code>ADMIN_TOKEN</code>	<code>{jobs:[{script_id, enabled}]}</code>
POST	<code>/cron/run</code>	admin if <code>ADMIN_TOKEN</code>	Tick now (<code>force</code> , optional <code>script_id</code>)



Deploy body

```
{
  "id": "btc-sma",
  "script": "//@version=6\nindicator(\"t\")\nplot(close)\nalertcondition(close>open,\"up\", \"green\"",
  "symbol": "BTCUSDT",
  "timeframe": "1d",
  "mode": "interpret",
  "data_source": "mock",
  "period": "6mo",
  "max_bars": 500,
  "enabled": true,
  "webhook_url": "https://hooks.example.com/pine",
  "forward_alerts": true
}
```

`data_source`: `mock` (deterministic, no network), `yahoo`, `ccxt` (`data_options.exchange`), `alphavantage`. Same provider factory as CLI `pyne data`.

Worked examples

Local (no Docker):

```
python -m backend.app    # :5002
pyne runner deploy script.pine --id demo --symbol BTCUSDT --source mock
pyne runner tick --id demo --force
pyne runner list
```

Same HTTP without the CLI:

```
curl -sS -X POST http://127.0.0.1:5002/scripts -H 'Content-Type: application/json' \
  -d '{"id":"demo","script":"//@version=6\nindicator(\"t\")\nplot(close)","data_source\":\"mock\",\"tin
curl -sS -X POST http://127.0.0.1:5002/cron/run -d '{"force":true}' -H 'Content-Type: application/
```

Docker:

```
PYNE_RUNNER=1 PYNE_RUNNER_SCHEDULER=1 docker compose up --build api
```

VPS: set the same env on `pynescrypt-api.service` (or the process environment `scripts/deploy_vps.sh` restarts). SQLite path should sit on a persistent disk (`PYNE_RUNNER_DB=/data/runner.db`).

Cloudflare Container: keep `PYNE_RUNNER=0` in `cf/src/index.ts` unless you want the runner inside the isolate; one gunicorn worker is already the default.



Failure modes

Symptom	Cause	Fix
<code>RUNNER_DISABLED</code> 404	Switch off	<code>PYNE_RUNNER=1</code>
<code>FORBIDDEN</code> 403	<code>ADMIN_TOKEN</code> set, header missing	<code>X-Admin-Token</code>
<code>skipped</code> / <code>no_new_bar</code>	Last close unchanged	Wait, or <code>force: true</code>
<code>fetch_failed</code>	yahoo/ccxt error	Use <code>mock</code> or fix <code>data_options</code>
Double ticks	Several gunicorn workers	<code>GUNICORN_WORKERS=1</code> or rely on flock

See also

- [POST /run](#)
- [Alerts](#)
- [pyne-worker cron](#) — edge equivalent
- [Docker](#)
- [App lifecycle](#)

OPTIMIZE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRYPT.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "POST /optimize" description: "Strategy hyperparameter search — N interpret Runtime runs with input.* overrides, TPE/random/grid, holdout or walk-forward."

POST `/optimize`

Abstract

`/optimize` searches Pine `input.*` values for a `strategy()` script. It loops the package **Runtime** (same interpret path as `/run`) with different overrides, scores closed trades, and returns the best assignment plus every trial.

This is **not** `/backtest/quick` (that route is a demo SMA simulator). It is **not** a Pine language surface — there is no `optimize.*` builtin.

AXIS Hyperparameter Optimisation calls this when the active engine is the Pro API (`server`). Pyodide / edge engines fall back to a client-side loop.

Conceptual model

```
flowchart TD
    REQ[POST /optimize] --> GATE[optional FREE_TIER_LIMITS bars / rate / slot]
    GATE --> KIND{strategy?}
    KIND -->|no| ERR[NOT_A_STRATEGY]
    KIND -->|yes| PARSE[parse once — host cache]
    PARSE --> LOOP[N trials]
    LOOP --> RUN[Runtime.run interpret + inputs]
    RUN --> SCORE[pair events → stats → objective]
    SCORE --> BEST[best OOS then IS]
```



Request

Field	Type	Required	Default	Notes
<code>script</code>	string	yes		Must declare <code>strategy(</code>
<code>data</code>	list	yes		Same OHLCV objects as <code>/run</code>
<code>space</code>	object	yes		<code>{ params: [{ name, kind, min, max, step?, choices? }] }</code>
<code>n_trials</code>	int	no	30	Capped at 200
<code>sampler</code>	string	no	<code>auto</code>	<code>auto</code> <code>random</code> <code>tpe</code> <code>grid</code> (<code>auto</code> → <code>random</code> if <code>N<20</code> else <code>TPE</code>)
<code>objective</code>	string	no	<code>composite</code>	<code>net_pnl</code> <code>profit_factor</code> <code>calmar</code> <code>composite</code>
<code>validation</code>	object	no	<code>{ mode: "holdout" }</code>	<code>holdout</code> <code>walk-forward</code> <code>in-sample</code>
<code>min_trades</code>	int	no	5	Trials below this score <code>-inf</code>
<code>seed</code>	int	no		RNG seed
<code>symbol</code>	string	no	<code>CHART</code>	
<code>oos_every_trial</code>	bool	no	true	Holdout: score every trial on the test slice
<code>fixed_inputs</code>	object	no	<code>{}</code>	Current <code>input.*</code> values for names not in <code>space</code> (trial params win on overlap)
<code>libraries</code>	list	no	<code>[]</code>	Same AXIS git-publish list as <code>/run</code>

`space.params[].kind`: `int` | `float` | `bool` | `categorical`. Numeric axes **require** `min` and `max`.

`validation` extras: `holdout_frac` (default 0.3), `trainBars` / `testBars` / `stepBars` for walk-forward.

Estimated engine runs are capped at 400 (`n_trials * folds`).



Success

```
{
  "status": "success",
  "sampler": "tpe",
  "objective": "composite",
  "validation": { "mode": "holdout", "holdout_frac": 0.3, "trainBars": 200, "testBars": 50, "step": 1,
  "n_trials": 30,
  "trials": [
    {
      "index": 0,
      "params": { "Fast": 8 },
      "is_score": 12.4,
      "oos_score": 3.1,
      "is_stats": { "total_pnl": 10, "trades": 8, "max_dd": 0.1 },
      "oos_stats": { "total_pnl": 3, "trades": 6, "max_dd": 0.08 },
      "error": null,
      "engine_runs": 2,
      "ms": 210.4
    }
  ],
  "best_index": 4,
  "best_params": { "Fast": 6 },
  "best_is_score": 15.0,
  "best_oos_score": 4.2,
  "engine_runs": 60,
  "ms": 8421.2,
  "warning": null
}
```

Full event arrays are **not** returned (memory). Re-run the winner via `/run + inputs`. Success includes `validation` and per-trial `oos_stats` / `engine_runs` / `ms`.

Errors

HTTP	code	When
400	MISSING_FIELD / INVALID_FIELD / UNKNOWN_FIELDS / INVALID_BODY	Schema — empty/ <code>{}</code> body, missing required fields, wrong types, extras
400	NO_SCRIPT / NO_DATA	<code>script</code> / <code>data</code> present but empty after schema
400	NOT_A_STRATEGY	Script is not <code>strategy()</code>
400	INVALID_SPACE	Missing bounds / empty params
400	INVALID_VALIDATION	<code>validation.mode</code> not <code>holdout</code> <code>walk-forward</code> <code>in-sample</code>
400	TOO_MANY_RUNS	Estimated runs exceed 400
400	GRID_TOO_LARGE	Grid sampler product exceeds 200 cells
413 / 429	same as <code>/run</code>	Free-tier caps — only when <code>FREE_TIER_LIMITS</code> is on



Invariants

1. Always **interpret** (input overrides skip compile).
2. Train and test are **bar slices**, not trade filters on a full-sample run.
3. In-sample mode is allowed and **warned**.
4. Ranking prefers **OOS** score when validation is holdout or walk-forward.

CLI

```
pyne optimize strat.pine --trials 20 --sampler tpe \  
  --space '{"params":[{"name":"Fast","kind":"int","min":3,"max":12}]}'  
# walk-forward: --validation walk-forward (derives train/test from --bars)  
# override: --train-bars 80 --test-bars 20 --step-bars 20
```

See also

- [POST /run](#)
- [Runtime bridge](#)
- [AXIS end-user guide: Hyperparameter Optimisation](#)

PREVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Preview Endpoints" description: "Pro chart and indicator thumbnail routes under /preview with usage tracking."

PREVIEW ENDPOINTS

Abstract

Preview routes render **PNG thumbnails** (base64) for desk UIs and docs. They require an API key via

`@track_usage`, accept OHLCV-shaped **objects** (column arrays), and do **not** run the full `Runtime` bar loop for

`/preview/chart` (data is plotted directly). `/preview/indicator` evaluates a small set of closed-form `ta.*`

expressions in pure Python for the series, then plots.



Conceptual model

```
flowchart LR
    KEY[API key] --> TRACK[track_usage]
    TRACK --> CHART["POST /preview/chart"]
    TRACK --> IND["POST /preview/indicator"]
    CHART --> R1[render_line_chart / render_ohlcvcv_chart]
    IND --> COMP[_compute_indicator]
    COMP --> R2[render_line_chart]
    R1 --> B64[base64 PNG]
    R2 --> B64
```

Blueprint: `preview_bp` with `url_prefix="/preview"` in `backend/api/preview.py`.

Interface surface

Auth

Both routes: `@track_usage` → must pass `require_api_key` and consume one call on success.

POST /preview/chart

Body (documented; schemas exist as `PREVIEW_CHART_SCHEMA`)

```
{
  "script": "optional string",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "options": {
    "type": "line",
    "color": "#2196F3",
    "show_volume": false,
    "width": 600,
    "height": 300
  }
}
```

Option	Default	Clamp
type	"line"	"ohlcvcv" selects candlestick renderer
color	#2196F3	line chart only
width	600	max 1200
height	300	max 600
show_volume	false	twin-axis volume on line chart

Success



```
{
  "status": "success",
  "chart": "<base64 png>",
  "meta": {
    "type": "line",
    "bars": 252,
    "last_value": 0,
    "first_value": 0,
    "change": 0,
    "change_pct": 0,
    "rendered_at": 0
  },
  "tier_info": {}
}
```

Errors

code	When
NO_DATA	Missing <code>data</code> (both preview routes)
NO_CLOSE_DATA	Empty <code>close</code> array (chart and indicator)
RENDER_ERROR	matplotlib / renderer exception (<code>/preview/chart</code> only; indicator parse/render exceptions fall back to plotting <code>close</code>)

Note: `script` is accepted for API symmetry but **not executed** on this path today — the chart is pure data visualization.

POST `/preview/indicator`

Body

```
{
  "expression": "ta.sma(close, 14)",
  "data": { "close": [/* ... */] },
  "options": { "color": "#FF9800", "width": 600, "height": 300 }
}
```

Supported expression prefixes in `_compute_indicator` :

Expression	Series
<code>ta.sma(..., period)</code>	SMA of close
<code>ta.ema(..., period)</code>	EMA
<code>ta.rsi(..., period)</code>	RSI
<code>ta.macd(...)</code>	MACD histogram (12/26/9)
other	falls back to raw <code>close</code>

Parse failures in the helper fall back to plotting `close` .



Internals

Path	Role
backend/api/preview.py	Route handlers + indicator math
backend/services/chart_renderer.py	PNG encode
backend/middleware/auth.py	track_usage

Schemas `PREVIEW_*` in `schemas.py` document intended validation; current handlers use `request.get_json()` or `{} directly.`

Invariants and edge cases

1. **Data shape differs from `/run`** : preview uses **columnar dicts**; `/run` uses **list of bar dicts**.
2. **Not a full Pine interpreter** for indicator expressions — string prefix parsing only.
3. **Width/height clamps** protect worker memory/CPU.
4. Successful responses include `tier_info` from the authenticated key.

Worked example

```
curl -s http://127.0.0.1:5002/preview/chart \  
-H "Authorization: Bearer $API_KEY" \  
-H 'Content-Type: application/json' \  
-d '{  
  "data": {"close": [1,2,3,4,5], "volume": [10,10,10,10,10]},  
  "options": {"type": "line", "show_volume": true}  
}' | jq '.meta'
```

Failure modes

Symptom	Cause
401	Missing key
429	Tier limit
Empty-looking PNG	Renderer empty-chart path when all values null

See also

- [Chart renderer](#)
- [Auth and keys](#)
- [Run](#)

BACKTEST

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Backtest Endpoint" description: "POST /backtest/quick — usage-tracked strategy metrics, equity curve, and optional mock OHLCV."

BACKTEST ENDPOINT

Abstract

`POST /backtest/quick` is a Pro, usage-tracked route that runs a **simplified** strategy simulation over columnar OHLCV (or generated mock bars), returns trade lists + summary metrics, and embeds a base64 equity-curve PNG. It is optimized for speed/demo UX; it is not a full fidelity substitute for event-aware `Runtime` strategy evaluation on `/run`.



Conceptual model

```
flowchart TD
    REQ[POST /backtest/quick] --> AUTH[track_usage]
    AUTH --> DATA{data or mock_data?}
    DATA -->|mock / empty close| MOCK[generate_mock_ohlc]
    DATA -->|provided| OHLCV[columnar OHLCV]
    MOCK --> SIM[run_quick_backtest]
    OHLCV --> SIM
    SIM --> METRICS[equity, trades, summary]
    SIM --> PNG[render_equity_curve]
```

Blueprint: `backtest_bp`, prefix `/backtest`.

Interface surface

Request

```
{
  "script": "@version=6\nstrategy(\"s\")...",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "initial_capital": 10000.0,
  "mock_data": false,
  "mockBars": 252
}
```

Field	Default	Notes
<code>script</code>	required	Empty → <code>NO_SCRIPT</code>
<code>data</code>	<code>{}</code>	Columnar; may be omitted if <code>mock_data</code>
<code>initial_capital</code>	<code>10000</code>	Starting equity
<code>mock_data</code>	<code>false</code>	Force synthetic bars
<code>mockBars</code>	<code>252</code>	Length of mock series

If neither usable `close` nor `mock_data` → `NO_DATA`.



Success

```
{
  "status": "success",
  "result": {
    "equity_curve": [10000, ...],
    "trades": [
      {
        "entry_time": 0,
        "entry_price": 0,
        "exit_time": 0,
        "exit_price": 0,
        "direction": "long",
        "pnl": 0,
        "pnl_pct": 0,
        "size": 1
      }
    ],
    "summary": {
      "total_pnl": 0,
      "total_pnl_pct": 0,
      "sharpe_ratio": 0,
      "max_drawdown": 0,
      "max_drawdown_pct": 0,
      "win_rate": 0,
      "profit_factor": 0,
      "total_trades": 0,
      "winning_trades": 0,
      "losing_trades": 0,
      "avg_win": 0,
      "avg_loss": 0
    },
    "equity_chart": "<base64 png>"
  },
  "tier_info": {},
  "meta": {
    "bars": 252,
    "initial_capital": 10000,
    "completed_at": 0
  }
}
```

Errors

code	HTTP	When
NO_SCRIPT	400	Empty script
NO_DATA	400	No data and not mock
BACKTEST_ERROR	500	Exception in simulation
UNAUTHORIZED / RATE_LIMITED	401 / 429	Auth

Internals

`run_quick_backtest` wraps `run_backtest(..., plot_chart=True)` and returns `BacktestResult.to_dict()` (`backend/services/backtest.py`). `avg_bars_in_trade` exists on the dataclass but is **not** exported in `summary`.



MVP simulation characteristics:

- 1. Optionally `parse(script)` (errors soft-ignored for MVP path).
- 2. Precompute long/short entry signals from dual SMA cross (10 vs 20) starting at bar 20.
- 3. Exit heuristics via RSI-like average and opposite signals / end of series.
- 4. Equity curve + trade PnL with optional commission/slippage parameters on the lower-level API.
- 5. Metrics: Sharpe ($\sqrt{252}$ scaling), max drawdown, win rate, profit factor.
- 6. Chart via `render_equity_curve`.

`generate_mock_ohlc(n_bars)` produces synthetic columns for demos.

Path	Role
<code>backend/api/preview.py</code>	<code>quick_backtest</code> route
<code>backend/services/backtest.py</code>	Simulation + metrics
<code>backend/services/chart_renderer.py</code>	Equity PNG

Invariants and edge cases

- 1. **Script content is lightly used** in the MVP sim — do not treat results as broker-accurate for arbitrary strategies. Prefer `/run` events for engine-faithful strategy traces.
- 2. **Columnar data**, same family as preview, not `/run` bar lists.
- 3. **Mock path** ignores incomplete user data when `mock_data` or empty close.
- 4. One usage increment per successful HTTP response under `track_usage`.

Worked example

```
curl -s http://127.0.0.1:5002/backtest/quick \
-H "Authorization: Bearer $API_KEY" \
-H 'Content-Type: application/json' \
-d '{
  "script": "@version=6\nstrategy(\"demo\")\n// body optional for MVP",
  "mock_data": true,
  "mock_bars": 120,
  "initial_capital": 25000
}' | jq '.result.summary'
```

Failure modes

Symptom	Cause
Implausible trades vs script logic	MVP signal model, not full evaluator
Empty trade list	No crosses in series / short history
Missing <code>equity_chart</code>	Renderer exception swallowed → empty string

See also

- [Runtime bridge](#) — faithful strategy events via `/run`



- Chart renderer
- Strategy builtins

CHART RENDERER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Chart Renderer" description: "Matplotlib Agg PNG services: line charts, OHLCV candles, equity curves, base64 encoding."

CHART RENDERER

Abstract

`backend/services/chart_renderer.py` is a headless plotting façade. It forces `matplotlib` into **Agg**, draws dark-theme charts for desk thumbnails, and returns **base64-encoded PNG** strings suitable for JSON transport. Callers are the preview and backtest routes; the module has no Flask dependency.



Conceptual model

```
flowchart LR
    VAL[series / OHLCV / equity] --> PLT[matplotlib Figure Agg]
    PLT --> BUF[BytesIO PNG]
    BUF --> B64[base64 utf-8 string]
    B64 --> API[JSON chart / equity_chart fields]
```

Interface surface

render_line_chart

Param	Default	Role
values	required	Y series; <code>None</code> → NaN gaps
dates	<code>None</code>	Optional x tick labels ($\leq \sim 6$ ticks)
title	"Chart"	Title string
color	#2196F3	Line + fill
height / width	300 / 600	<code>width</code> → figsize inches (<code>width/100</code>); height is unused for figsize — line charts use 3.0" (3.5" with volume), OHLCV 3.5", empty 2.5". dpi 100
show_volume	false	Twin axis bars from <code>ohlcv["volume"]</code>
ohlcv	<code>None</code>	Volume source when enabled

Empty / all-null values → `_render_empty_chart` placeholder.

render_ohlcv_chart

Candlestick + volume subplot pair from dict keys `open`, `high`, `low`, `close`, `volume`. Green/red body colors (`#4CAF50` / `#F44336`). Shared x-axis. Figsize (`width/100`, `3.5`) — the `height` argument is ignored.

render_equity_curve

Single series with profit/loss fill relative to zero baseline and a dashed reference at the first equity point. Title fixed "Equity Curve".

Return type

All public renderers → `str` (base64 PNG without data-URI prefix). Clients typically use:

```
data:image/png;base64,{chart}
```



Internals

Detail	Implementation
Backend	<code>matplotlib.use("Agg")</code> before <code>pyplot</code> import
Theme	<code>fig #1E1E1E</code> , axes <code>#252526</code> , muted grids
Cleanup	<code>plt.close(fig)</code> after <code>savefig</code>
Empty state	Centered “No data available” text

Path	Role
<code>backend/services/chart_renderer.py</code>	Renderers
<code>backend/api/preview.py</code>	Consumers
<code>backend/services/backtest.py</code>	Equity chart hook

Invariants and edge cases

1. **Not interactive** — no GUI backend; safe for servers.
2. **NaNs** break lines at gaps (numpy nan).
3. **Candle body height** uses `+ 1e-9` to keep zero-range bodies visible.
4. **Thread safety**: matplotlib global state is historically process-local; multi-threaded gunicorn workers should avoid concurrent pyplot use on one worker or serialize renders.
5. Exceptions in callers often become HTTP `RENDER_ERROR` or empty equity chart strings.

Worked example

```
from backend.services.chart_renderer import render_line_chart

b64 = render_line_chart([1.0, 1.2, 1.1, 1.4], title="demo", color="#A3E635")
assert isinstance(b64, str) and len(b64) > 100
```

Failure modes

Symptom	Cause
ImportError Agg	Incomplete matplotlib install
Huge base64	High width/height — routes clamp preview sizes
Blank image	Empty input path succeeded with placeholder

See also

- [Preview endpoints](#)
- [Backtest](#)

RUNTIME BRIDGE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Runtime Bridge" description: "pynescrypt.runtime.Runtime — package SoT bar-loop evaluate façade, PineSeries context, compile mode, and result packaging."

RUNTIME BRIDGE

Abstract

Package Runtime SoT: the bar-loop host, series, and `CustomEvaluator` live under `pynescrypt.runtime` (`src/pynescrypt/runtime/host.py`). The Pro API still imports `backend.runtime.Runtime`, which **re-exports** the package implementation (`sys.modules` alias) so monorepo and wheel installs share one host.

`Runtime` owns symbol metadata namespaces (`syminfo`, `timeframe`, `barstate`, ...), walks OHLCV bars, updates `PineSeries` for OHLC, visits a shared `CustomEvaluator` on the parsed AST each bar, drains strategy events, and packages multi-plot series for AXIS. Compile mode optionally swaps the AST walker for a Numba-backed subset



engine. Optional `timeout_seconds` provides a wall-clock circuit breaker on the interpret path (checked every 32 bars) for edge/cron budgets. Flask `/run` and `/run/batch` accept the same field (omit `/null / ≤ 0` → no timeout).

This is the bridge between Flask handlers and the language core — not a second semantics.

Conceptual model

```
flowchart TB
    SRC[Pine source] --> PARSE[parse mode=exec]
    OHLCV[bar list] --> LOOP[for each bar]
    PARSE --> LOOP
    LOOP --> SER[PineSeries.update OHLC]
    LOOP --> CTX[context bar_index time barstate]
    LOOP --> FILL[process_pending_orders]
    LOOP --> VISIT[evaluator.visit tree]
    VISIT --> PLOTS[plot_outputs]
    VISIT --> EV[strategy events]
    VISIT --> AL[alert firings]
    LOOP --> PACK[series map + plot_meta + drawings + alerts]
```

Interface surface

Construction

```
Runtime(symbol: str = "AAPL", run_id: str | None = None)
```

- Sets `syminfo.tickerid` / `name` / `prefix` (prefix from `EXCHANGE:SYMBOL` form).
- Generates `run_id` as `uuid4().hex[:16]` when omitted.

Helpers: `configure_footprint`, `update_bid_ask`.



```
run(source_code, ohlcv_data, data_feed=None, data_provider=None, mode="interpret", ...)
```

Arg	Role
source_code	Pine text
ohlcv_data	list[dict] with open/high/low/close/time[/volume/bid/ask]
data_feed / data_provider	request.* wiring; auto-resolved from chart bars when unset
mode	"interpret" (default for direct calls) · "compile" · "auto" (Pro API schema default)
timeout_seconds	Optional wall-clock budget (seconds); interpret path only — partial results + timed_out + error_kind=runtime when exceeded. Flask /run and /run/batch accept it (omit / ≤ 0 = no budget)
libraries	Optional [{namespace, name, version, source}] registered via register_library_source before import ns/Name/ver. mode=auto forwards libs into interpret fallback. Flask /run caps at 32
inputs / profiler	input.* overrides (interpret); non-empty inputs or profiler force interpret under auto (compile_fallback_reason)
realtime_*	Optional host simulation for varip multi-tick tests (realtime_last_bar, realtime_ticks, realtime_bars, realtime_from_bar)

Interpret path (default)

1. Resolve request sources (non-fatal on failure).
2. parse(source_code, mode="exec") — on failure { "error": "Parse Error: ..." }.
3. Init PineSeries for OHLC + context dict (timeframe daily defaults, barstate, chart colors).
4. Construct CustomEvaluator(context=..., data_feed=..., data_provider=...), reset var declarations + drawing registry + clear_alerts().
5. Per bar:
 - Update series + calendar fields from timestamp
 - Update barstate flags (isfirst / islast / history confirmed)
 - process_pending_orders when available
 - evaluator.visit(tree) — on failure { "error": "Runtime Error at bar ..." }
 - Lock pine defs after first bar (_pine_defs_locked) to avoid multi-dispatch blow-ups
 - Drain strategy events; stamp script_id / run_id
 - Capture plot outputs
6. Build series / plot_meta from all plot indices (disambiguate duplicate titles with _2 suffixes).
7. Export drawings via DrawingRegistry.export_for_api(bar_times).
8. Export alerts via export_alerts_from_evaluator (optional alert_conditions).

Return keys: plots, series, plot_meta, events, drawings, alerts, count, script_id, run_id, mode (+ optional alert_conditions, timed_out, error_kind).

mode=auto

1. Non-empty inputs → interpret immediately (compile_fallback_reason = "input.* overrides require interpret path").
2. _compile_eligible: reject top-level import and any request.* token (cached per source hash).



3. Eligible → `_run_compiled` . On success set `auto_backend=compile` . On compile/runtime error, fall back to interpret and set `compile_fallback_reason` .
4. Ineligible → interpret + `compile_fallback_reason` (e.g. "import statements not supported in compile path").

Value mismatch never triggers a backend switch — that is the parity harness's job.

Compile path

1. Import `pynescrypt.compiler.engine` — missing → `error_kind=compile` .
2. **Do not** require `has_numba()` . Object-mode scripts (strategy, UDT, drawings) compile to a Python/numpy loop. Missing Numba only fails pure-numeric emit (`CompileNumbaRequiredError`); `auto` caches that failure.
3. Host LRU by raw-source sha256 → `compile_script` on miss.
4. Pack OHLCV to float64 columns **including** `time=` bar-open ms (`_ohlcv_pack_cached`).
5. `compiled.run(opens, highs, lows, closes, volumes, time=times)` .
6. Lift `__drawings` / `__events` ; GC compile drawings by declaration caps; merge visual series (bgcolor / plotshape / ...) into titled keys.
7. Envelope: `mode: "compile"` , `alerts: []` , `object_mode` , `compile_cached` , `compile_ms` . `generated_code` only when `PYNESCRIPT_RETURN_GENERATED_CODE=1` .

PineSeries (`pynescrypt.runtime.series`)

History deque (default maxlen **1000**, raised by `max_bars_back` / `PYNE_SERIES_MAX`). `PYNE_SERIES_CAP` default **on** (list-length trim). `PYNE_SERIES_RING` default **off** (chronological O(1) lookback when `1`). `series[0]` current / `series[n]` lookback, arithmetic on **current** values, `None` propagates as na-like absence. Negative indices raise — Pine Script™ does not allow them. `backend.series` re-exports the package type.

Context namespaces

Defined on the host: `Syminfo` , `Chartinfo` , `Timeframe` , `Barstate` , `Chart` — attribute names aim at Pine Script™ v5–v6 surface (`timeframe.isdaily` , `syminfo.isin` , ...).

Internals

Path	Role
<code>src/pynescrypt/runtime/host.py</code>	Package SoT <code>Runtime</code> bar loop
<code>src/pynescrypt/runtime/evaluator.py</code>	<code>CustomEvaluator</code> specialization
<code>src/pynescrypt/runtime/series.py</code>	<code>PineSeries</code>
<code>backend/runtime.py</code>	Compat shim → <code>pynescrypt.runtime</code>
<code>backend/evaluator.py</code> / <code>backend/series.py</code>	Compat shims
<code>src/pynescrypt/ast/helper.py</code>	<code>parse</code>
<code>src/pynescrypt/ast/evaluator/**</code>	Builtin + strategy semantics
<code>src/pynescrypt/compiler/engine.py</code>	Compile mode

Flask wiring: `backend/app.py` constructs a **fresh** `Runtime` per request (and per batch job) so `run_id` and drawing registries do not leak across users.



Invariants and edge cases

- 1. **Parse once, visit many** — AST is not re-parsed per bar.
- 2. **Defs locked after bar 0** — performance invariant for large function tables.
- 3. **Drawing registry reset** at run start — isolation between requests.
- 4. **Primary `plots` list** is plot index 0 for backward compatibility; AXIS should prefer `series` + `plot_meta`.
- 5. **Errors are dicts**, not exceptions, for control-flow at the HTTP boundary (`"error"` in `result`).
- 6. Compile mode supports a **subset**; `mode=auto` falls back to interpret with `compile_fallback_reason` rather than failing the request.

Worked example

```
from pynescrypt.runtime import Runtime
# or: from backend.runtime import Runtime # Pro API monorepo path

rt = Runtime(symbol="NASDAQ:AAPL")
out = rt.run(
    '//@version=6\nindicator("x")\nplot(close, "C")',
    [{"time": i, "open": 1, "high": 2, "low": 0.5, "close": 1.0 + i * 0.01, "volume": 1} for i in range(50)]
)
assert "error" not in out
assert out["count"] == 50
assert "C" in out["series"] or "plot_0" in out["series"]
```

Failure modes

Message prefix	Cause
Parse Error:	Grammar / syntax
Runtime Error at bar	Evaluator exception
Order fill error at bar	Broker sim pending orders
Compile mode requires numba / CompileNumbaRequiredError	Numeric emit without Numba (object-mode scripts still run)
Compile Error: / Compiled Runtime Error:	Engine path (<code>error_kind=compile</code> / <code>runtime</code>)
Script execution timed out	<code>timeout_seconds</code> exceeded (<code>timed_out</code> , <code>error_kind=runtime</code>)

See also

- [Contract](#)
- [Run endpoint](#)
- [Series and history](#)
- [Strategy events](#)

CONTRACT

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED

THIS FILE IS PART OF PYNESCRYPT.

PYNESCRYPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY

IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY

THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR

(AT YOUR OPTION) ANY LATER VERSION.

PYNESCRYPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL, BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.

YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE

ALONG WITH PYNESCRYPT. IF NOT, SEE

[HTTPS://WWW.GNU.ORG/LICENSES/](https://www.gnu.org/licenses/).

SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Evaluate Contract" description: "Shared request/response evaluate contract across Flask Pro API, pyne-worker, and in-process Runtime.run (Python + PyneTS)."

EVALUATE CONTRACT

Abstract

The **evaluate contract** is the JSON dialect that lets AXIS, HOOX, CLI tools, and edge workers call *any* PYNE-compatible host interchangeably. Flask `POST /run` is the reference implementation; Python **pyne-worker** aims for the same HTTP fields. **PyneTS** speaks the in-process form (`RuntimeResult`).

This page freezes the semantic envelope — not transport auth headers.



Conceptual model

```
flowchart LR
    subgraph Producers
        AXIS[AXIS]
        CLI[CLI / scripts]
        HOOX[HOOX mesh]
    end
    subgraph Hosts
        FLASK[Flask /run]
        PYW[pyne-worker]
        TS[PyneTS Runtime.run]
    end
    AXIS --> FLASK
    AXIS --> PYW
    CLI --> FLASK
    CLI --> TS
    HOOX --> PYW
```

Invariant: given the same script + OHLCV + mode, hosts should agree on plot series and strategy events within known parity limits (see [numerical validation](#) and [PyneTS parity](#)). AXIS engines call Python hosts today, not PyneTS.

Interface surface

Request (single evaluate)

Field	Type	Required	Description
script	string	yes	Full Pine source
data	Bar[]	yes	Chronological OHLCV (ohlcv alias on pyne-worker)
symbol	string	no	Default CHART / host-specific
mode	"interpret" "compile" "auto"	no	Runtime.run default when omitted: PYNE_RUNTIME_MODE or interpret .Flask RUN_SCHEMA default auto
inputs	object	no	input.* overrides keyed by title (forces interpret under auto)
libraries	{namespace, name, version, source}[]	no	In-process import ns/Name/ver .Flask /run and /run/batch accept this (max 32).
profiler	bool	no	Per-line interpret timings (forces interpret)
timeout_seconds	number	no	Optional wall-clock interpret budget (checked every 32 bars; sets timed_out). Flask /run and /run/batch accept it; omit / null / ≤ 0 → no timeout.
webhook_url	string	no	Flask L2 alert webhook (overrides ALERT_WEBHOOK_URL)
data_source	string	no	Optional external history provider id (free paths: chart / mock only)
data_options	object	no	Provider configuration

Bar



Field	Type	Required	Notes
time	number	recommended	Unix ms preferred
open	number	yes	
high	number	yes	
low	number	yes	
close	number	yes	
volume	number	no	Default host-dependent
bid / ask	number	no	Runtime bid/ask hooks

Response (success)

Field	Type	Description
status	"success"	Flask sets this; workers may omit and use HTTP 200 only — prefer including it
plots	(number null)[]	Primary series (plot 0)
series	Record<string, (number null)[]>	Named multi-series
plot_meta	Record<string, PlotMeta>	Display metadata
events	StrategyEvent[]	Strategy / trade-like events
drawings	Drawing[]	line/label/box export
alerts	AlertEvent[]	alert() / true alertcondition() firings (interpret; [] on compile)
alert_conditions	object[]	optional condition evaluations
alert_forward	object	optional L2 webhook delivery summary
count	number	Bars evaluated
script_id	string	Stable hash prefix of source
run_id	string	Per-invocation id
mode	string	Effective mode (interpret compile)
auto_backend	string	auto only: which path ran
compile_fallback_reason	string	auto only: why compile was skipped or failed
object_mode	bool	Compile path: pure-Python bar loop (no njit)
timed_out	bool	Interpret budget exceeded (Runtime.run(timeout_seconds=...))
data_source	string	Flask echo of resolved source label

PlotMeta (Flask)



```
{
  "title": "C",
  "color": "#2196F3",
  "linewidth": 1,
  "index": 0,
  "kind": "plot"
}
```

`kind` is `plot` | `hline` | `fill` | `bgcolor` | `plotshape` | `plotchar` | `plotarrow`. Fill bands add `plot1` / `plot2` title refs; `hline` may add `price`. Extra style keys (`style`, `linestyle`, `location`, `text`, `char`, `size`) appear when the interpret collector has them.

StrategyEvent (minimum)

Hosts should preserve:

```
{
  "type": "string",
  "bar_index": 0,
  "script_id": "...",
  "run_id": "..."
}
```

Additional fields follow evaluator `to_dict()` / TS port parity — treat unknown keys as forward-compatible.

AlertEvent (minimum)

```
{
  "message": "string",
  "freq": "once_per_bar",
  "bar_index": 0,
  "time": 0,
  "source": "alert",
  "script_id": "...",
  "run_id": "..."
}
```

See [Alerts](#) for frequency rules and webhooks.

Response (failure)

Flask `/run`:

```
{
  "status": "error",
  "code": "EXECUTION_ERROR",
  "message": "Runtime Error at bar ...",
  "error_kind": "runtime"
}
```

Optional `error_type`, `error_bar`, `logs`, `profile`, `meta` pass through when `Runtime` set them. `error_kind` is `parse` | `compile` | `runtime` | `data` | `order` | `mode`.

Workers may use equivalent `{ error: string }` or HTTP 4xx/5xx with message body. Clients should accept:



1. Envelope `status === "error"` with `message` , or
2. Presence of top-level `error` string (raw `Runtime.run` shape).

Batch evaluate (Flask extension)

POST `/run/batch` :

Request: `scripts: (string | {id, script})[]` (max 8) + shared `data` + optional `symbol` / `data_source` / `data_options` / `mode` / `profiler` / `libraries` / `timeout_seconds` / `webhook` fields. **Not** on the batch schema: `inputs` .

Response:

```
{
  "status": "success" | "partial",
  "results": [ /* per-script success or error object with id */ ],
  "count": 0,
  "ok": 0,
  "data_source": "chart"
}
```

Workers may implement batch as N single evaluates; AXIS should not assume batch exists on every host.

Internals — reference mapping

Contract field	Flask source
<code>plots / series / plot_meta</code>	<code>pynescrypt.runtime.host.Runtime.run</code> packing loop
<code>events</code>	<code>evaluator._strategy_state.drain_events()</code> → <code>to_dict()</code> (compile: <code>__events</code>)
<code>drawings</code>	<code>DrawingRegistry.export_for_api</code> (compile: <code>__drawings</code>)
<code>alerts</code>	<code>export_alerts_from_evaluator</code> (interpret; compile returns <code>[]</code>)
<code>alert_forward</code>	<code>backend/alert_forwarder.maybe_forward_run_alerts</code>
<code>script_id</code>	<code>sha256(source)[:16]</code>
<code>run_id</code>	<code>Runtime._run_id</code>
<code>compile_fallback_reason</code>	<code>Runtime._run_auto</code>
<code>timed_out</code>	Interpret bar loop vs <code>timeout_seconds</code> (Flask <code>/run</code> and <code>/run/batch</code> when the field is set and <code>> 0</code>)

Schema enforcement on Flask free tier: `RUN_SCHEMA` / `RUN_BATCH_SCHEMA` in `backend/middleware/schemas.py` (strict, reject extras). Webhook fields: `webhook_url` , `forward_alerts` , `alert_last_bar` , `alert_batch` . `libraries` and `timeout_seconds` are on both `/run` and `/run/batch` .

Tests: `tests/test_backend.py` (`test_run_success` , `test_run_exports_alerts` , webhook cases), `tests/test_alert_forwarder.py` .



Divergences to remember

Surface	Divergence
Preview / backtest	Columnar <code>data</code> dict, not <code>Bar[]</code> ; not the evaluate contract
Compile mode	Extra keys <code>object_mode</code> , <code>compile_cached</code> , <code>compile_ms</code> , <code>nopython_fallback_reason</code> . <code>generated_code</code> only if <code>PYNESCRIPT_RETURN_GENERATED_CODE=1</code>
<code>timeout_seconds</code>	In-process <code>Runtime.run</code> , edge workers, and Flask <code>/run</code> / <code>/run/batch</code> (optional; omit = no budget)
Health	Flask <code>GET /</code> and <code>GET /health</code> (CORS-free); not part of evaluate
Auth	Flask free <code>/run</code> open (bar/rate/concurrency caps); workers may require HOOX mTLS / API gateways
Error codes	Flask uses <code>code</code> enums; workers should map to stable strings when possible

Invariants and edge cases

1. **Bar order is chronological ascending** — hosts do not sort for you.
2. `na` / **missing** appear as JSON `null` in series arrays.
3. **Duplicate plot titles** get suffixes (`_2` , ...) on Flask; clients should key on returned map keys, not assumed titles.
4. **Idempotent script_id** for identical source text across hosts.
5. **run_id is not stable** across retries — use for log correlation only.
6. Parity is best-effort on floating edges; pin fixtures when testing hosts against each other.

Worked example — host-agnostic client sketch

```
type EvaluateRequest = {
  script: string;
  data: Array<{
    time: number;
    open: number;
    high: number;
    low: number;
    close: number;
    volume?: number;
  }>;
  symbol?: string;
  mode?: "interpret" | "compile" | "auto";
  libraries?: Array<{
    namespace: string;
    name: string;
    version: number;
    source: string;
  }>;
};

type EvaluateSuccess = {
  plots: Array<number | null>;
  series: Record<string, Array<number | null>>;
  plot_meta: Record<string, Record<string, unknown>>;
  events: Array<Record<string, unknown>>;
  drawings: Array<Record<string, unknown>>;
  alerts: Array<Record<string, unknown>>;
  count: number;
  script_id: string;
  run_id: string;
  mode: string;
  auto_backend?: "compile" | "interpret";
  compile_fallback_reason?: string;
};
```

POST that body to Flask `/run` or the worker’s evaluate route; branch on `status` / `error`.

Failure modes

Client bug	Symptom
Sending columnar preview data to <code>/run</code>	Schema <code>data</code> must be a list
Assuming batch on worker	404 / unknown field
Ignoring <code>series</code> and only reading <code>plots</code>	Multi-plot scripts look single-series

See also

- [Run endpoint](#)
- [Runtime bridge](#)
- [pyne-worker](#)
- [pyne-agent-worker](#)



- Pro API usage (end user)
- Events